CS2201 Computer Organization & Architecture

L19: Cache Block Mapping and Identification



John Jose

Associate Professor

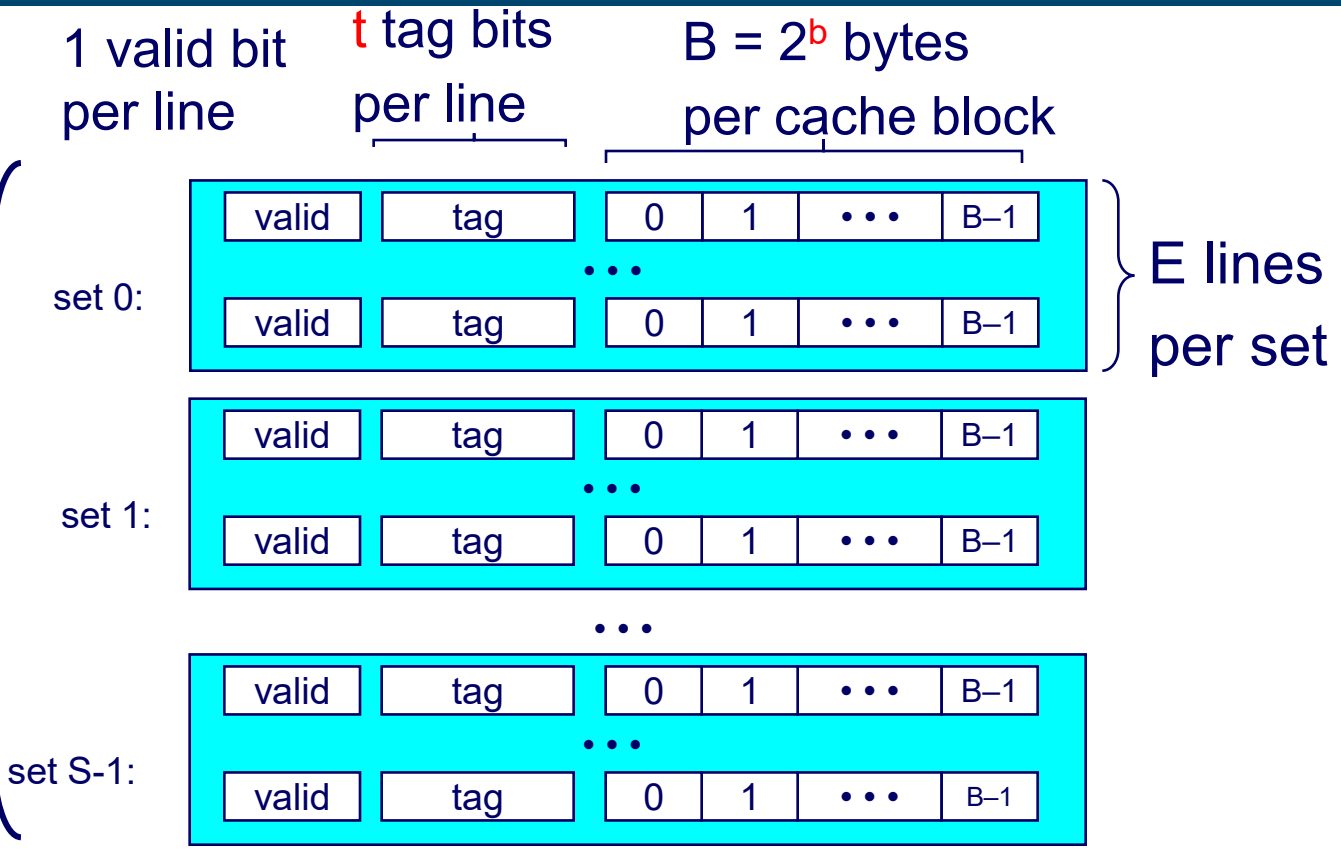
Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

General Organization of a Cache

- Cache is an array of sets
- Each set contains one or more lines
- Each line holds a block of data

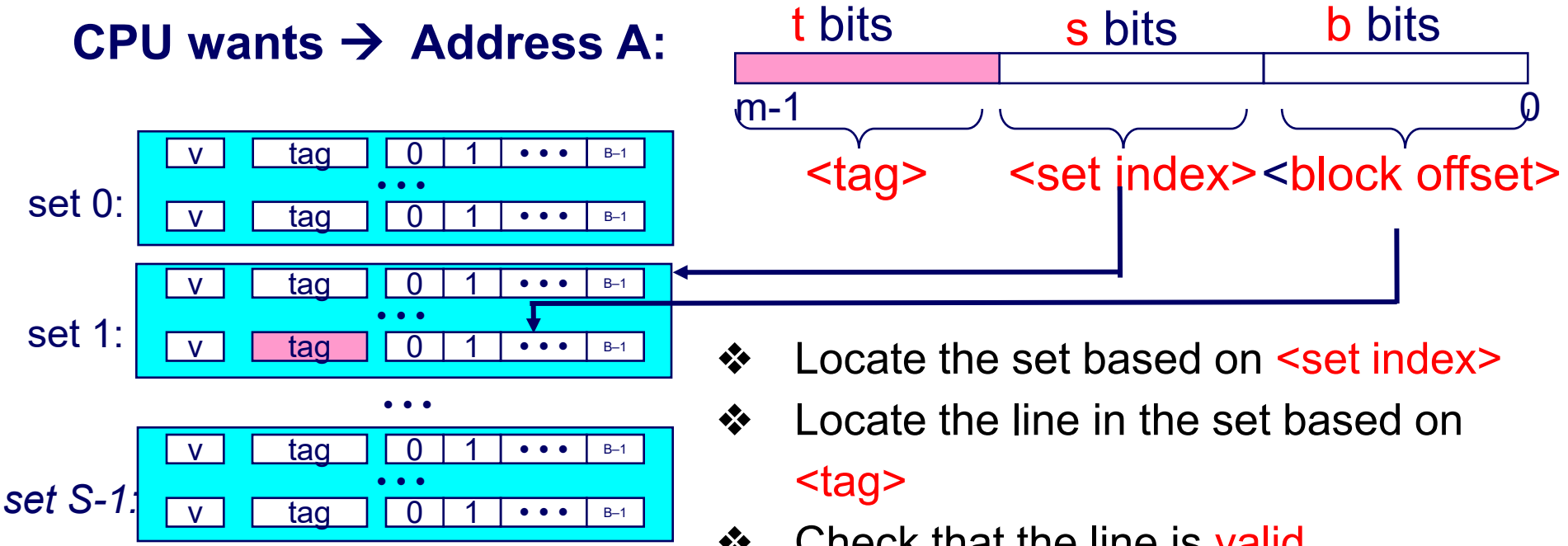
$S = 2^s$ sets



Cache size: C = B x E x S data bytes

Addressing Caches

CPU wants → Address A:

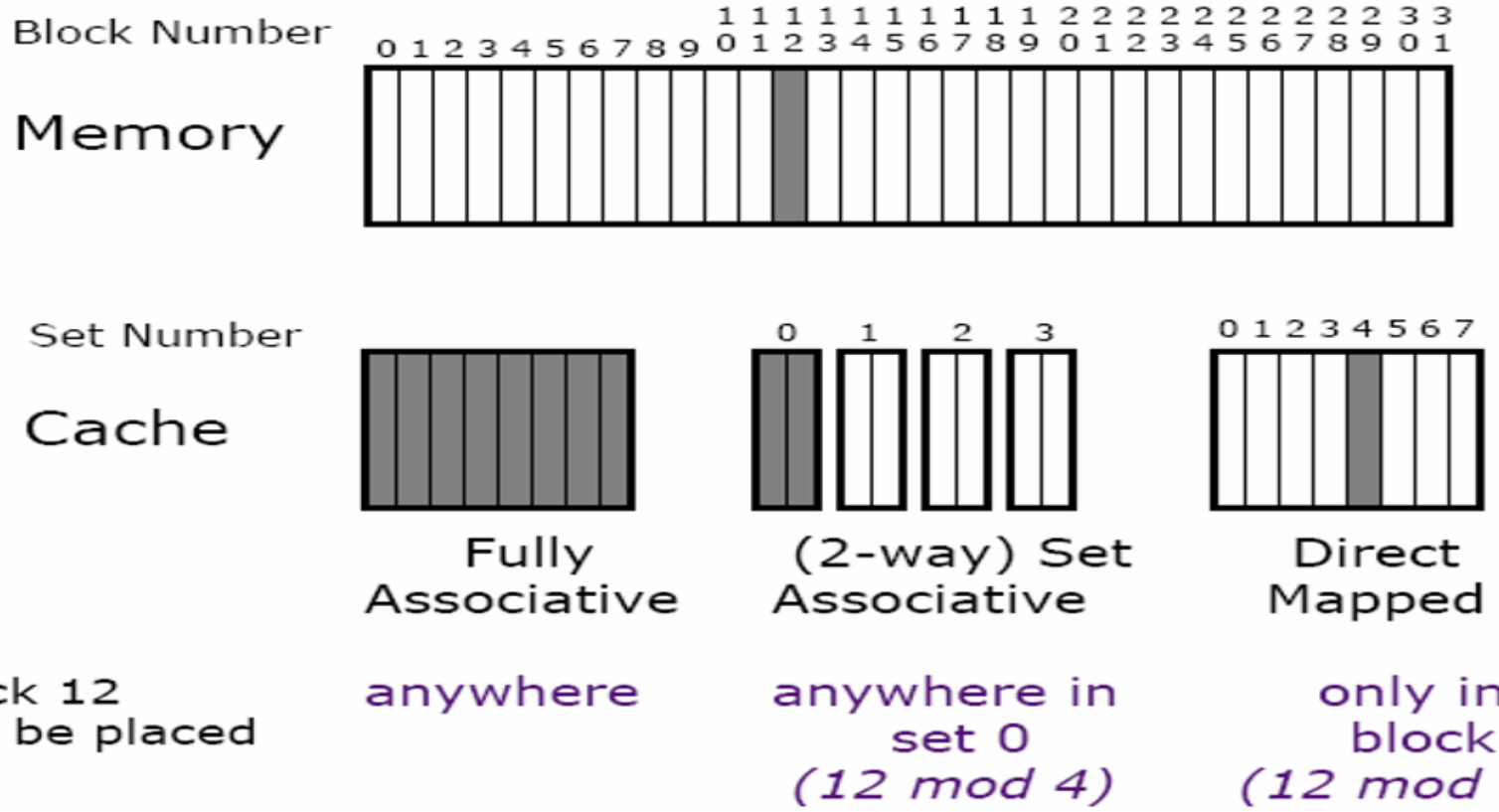


- Locate the set based on <set index>
- Locate the line in the set based on <tag>
- Check that the line is valid
- Locate the data in the line based on <block offset>

Four cache memory design choices

- Where can a block be placed in the cache?
  - **Block Placement**
- How is a block found if it is in the upper level?
  - **Block Identification**
- Which block should be replaced on a miss?
  - **Block Replacement**
- What happens on a write?
  - **Write Strategy**

Block Placement



Cache Mapping / Block Placement

- **Direct mapped**
  - Block can be placed in only one location
  - (Block Number) **Modulo** (Number of blocks in cache)
- **Set associative**
  - Block can be placed in one among a list of locations
  - (Block Number) **Modulo** (Number of sets)
- **Fully associative**
  - Block can be placed anywhere

Index and Offset Calculations

A cache has 512 KB capacity, 4B word, 64B block size and 8-way set associative. The system is using 32 bit address. Given the address 0X ABC89984, which set of cache will be searched and specify which word of the selected cache block will be forwarded if it is a hit in cache?

sets = CS/(BSxA) = $2^{19}/(2^6 x 2^3) = 2^{10}$ = 1024 sets

1 word = 4B , Hence 64 byte block has 16 words

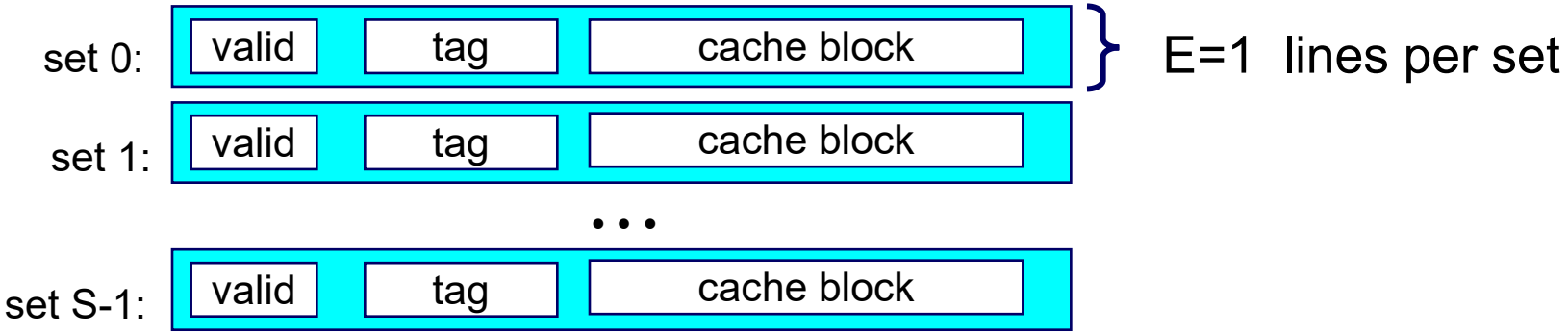
Tag = 16	Index =10	Offset=6 (4+2)

0x ABC89984 = 1001 1001 1000 0100 → Set 614, word 1

0x 485669AC = 0110 1001 1010 1100 → Set 422, word 11

Direct-Mapped Cache

- Simplest kind of cache, easy to build
- Only 1 tag compare required per access
- Characterized by exactly one line per set.

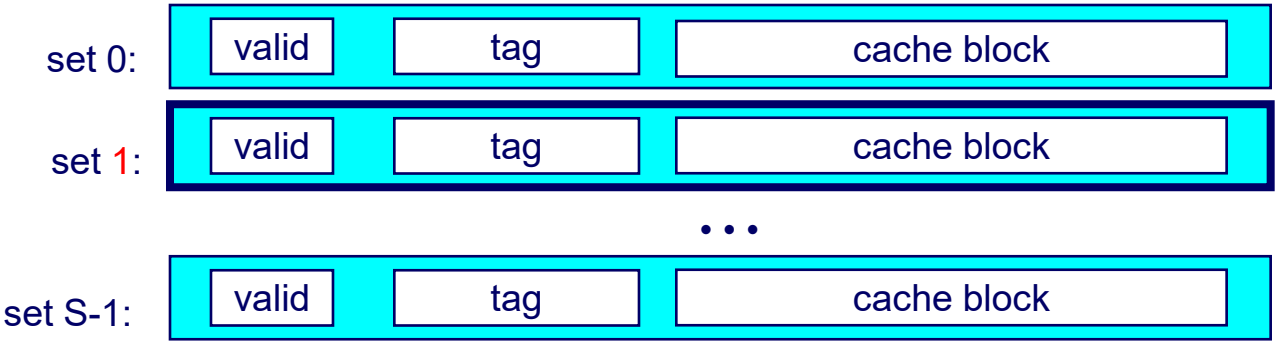


Cache size: C = B x S data bytes

Accessing Direct-Mapped Caches

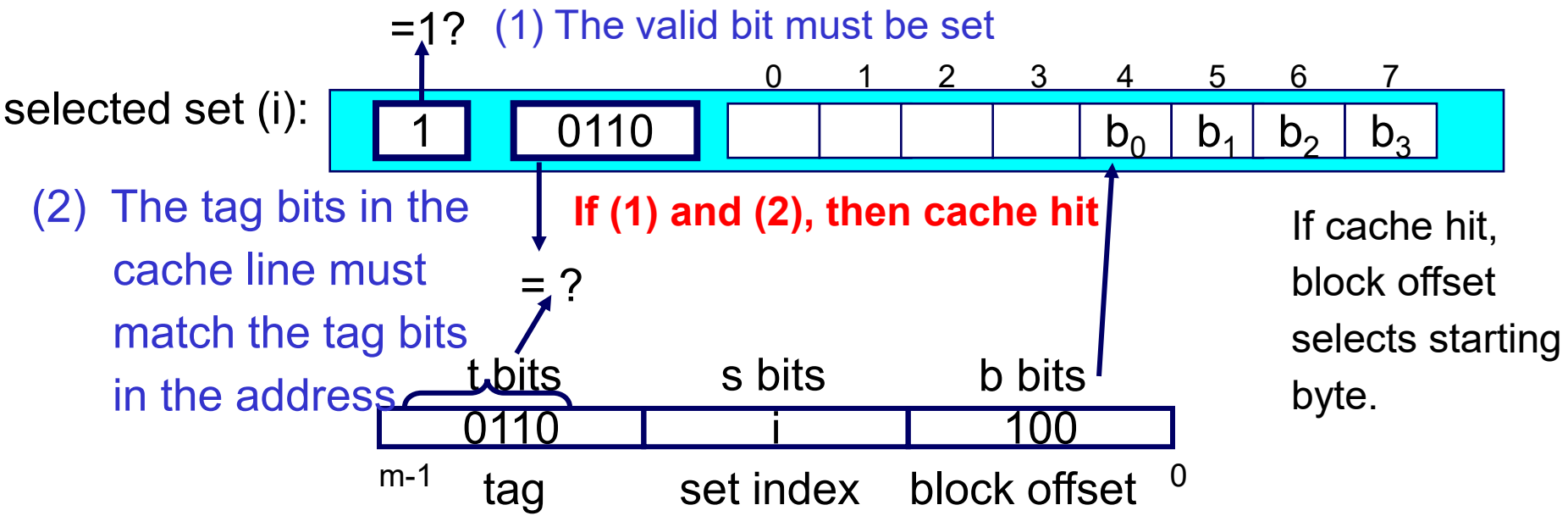
- Set selection is done by the set index bits



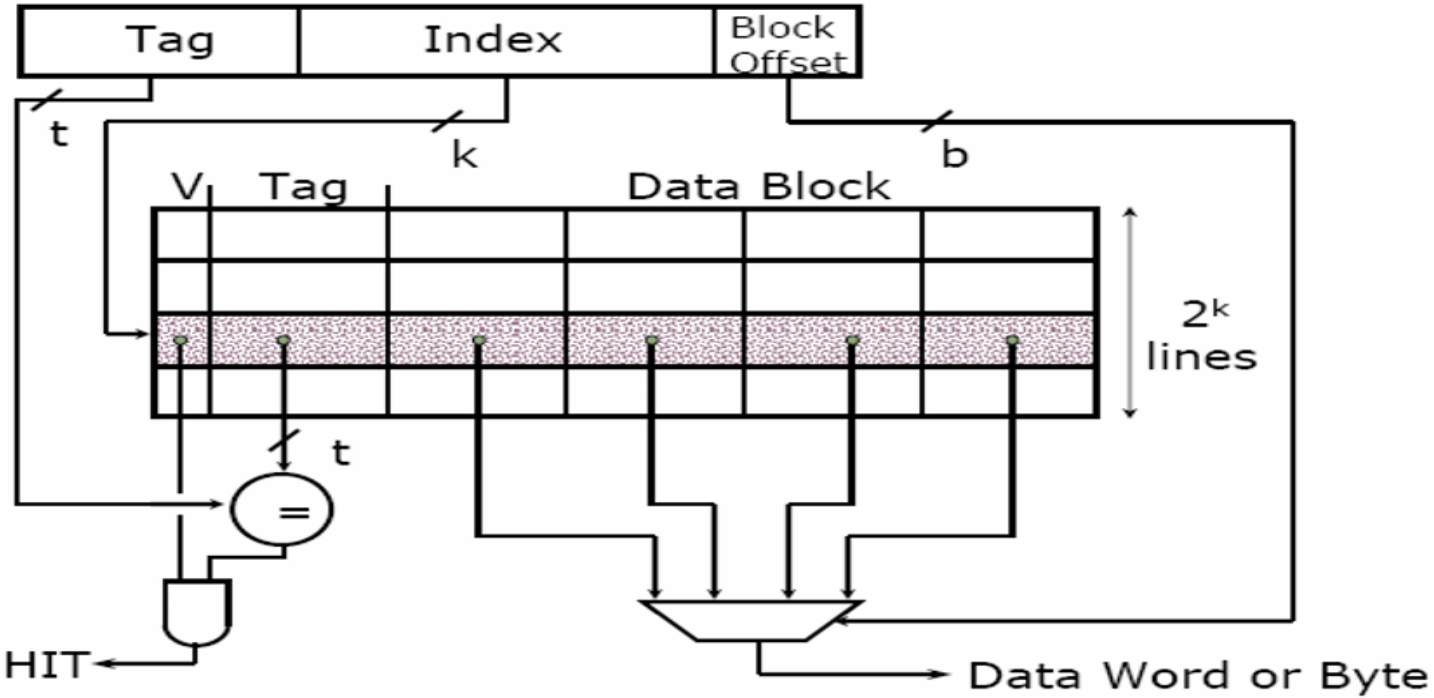


Accessing Direct-Mapped Caches

- Block matching: Find a valid block in the selected set with a matching tag
- Word selection: Then extract the word

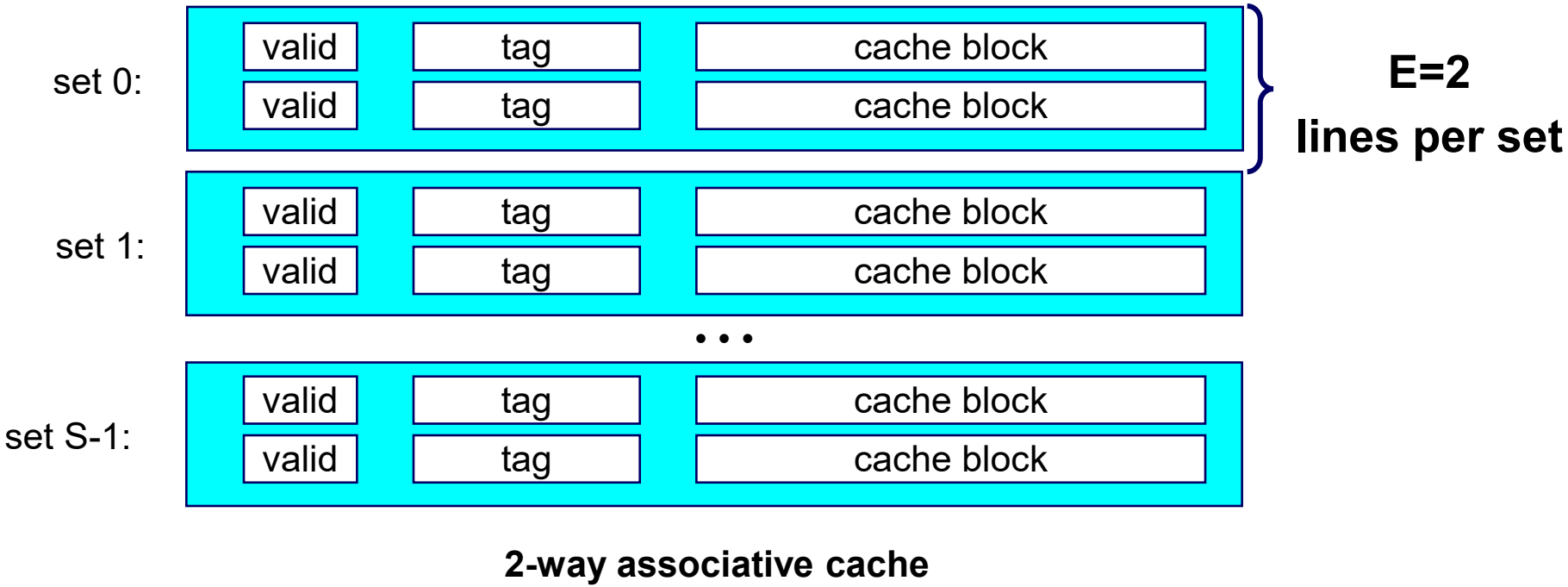


Block Identification – Direct mapped



Set Associative Cache

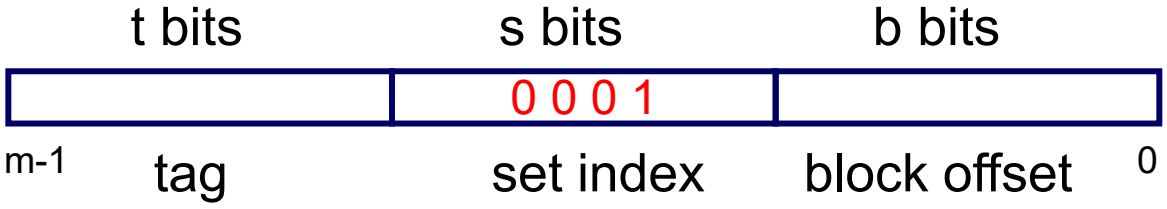
- Characterized by more than one line per set

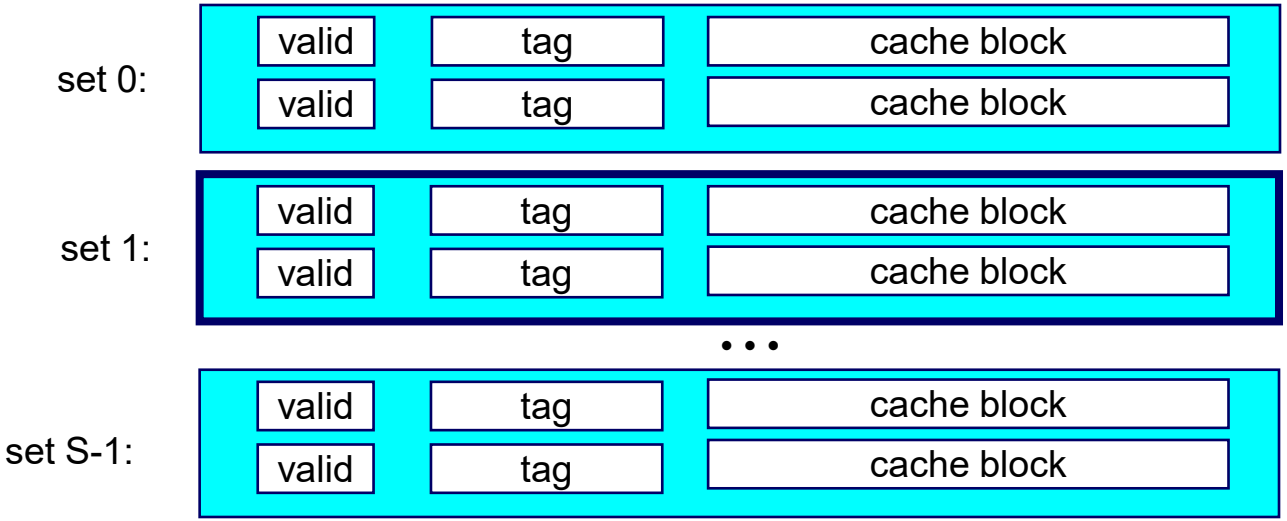


2-way associative cache

Accessing Set Associative Caches

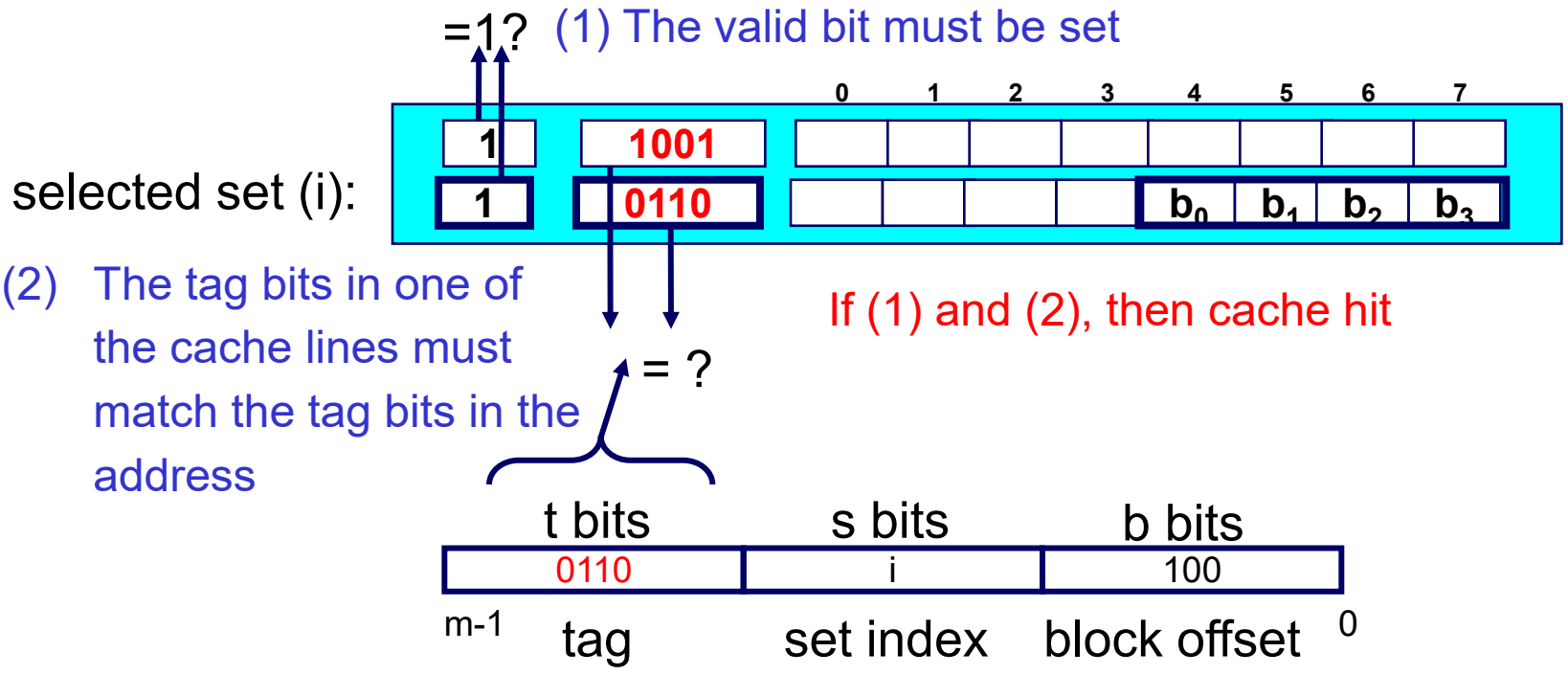
- Set selection is identical to direct-mapped cache





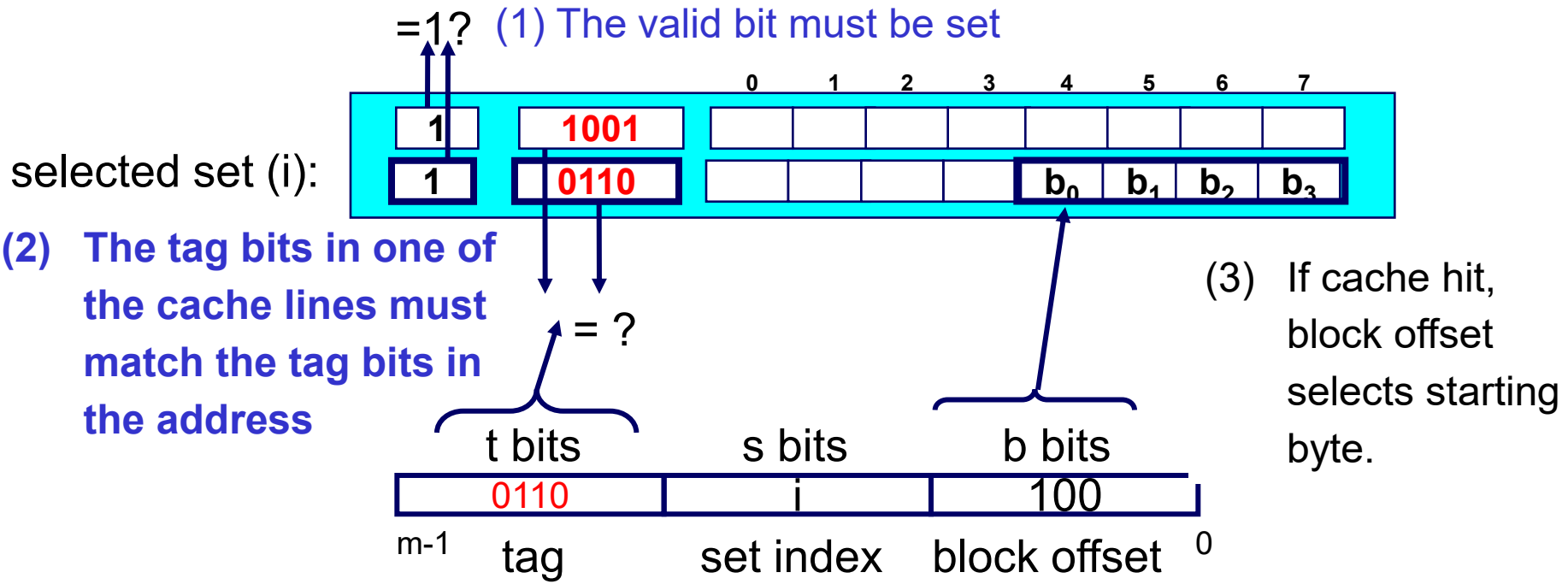
Accessing Set Associative Caches

- Block matching is done by comparing the tag in each valid line in the selected set.

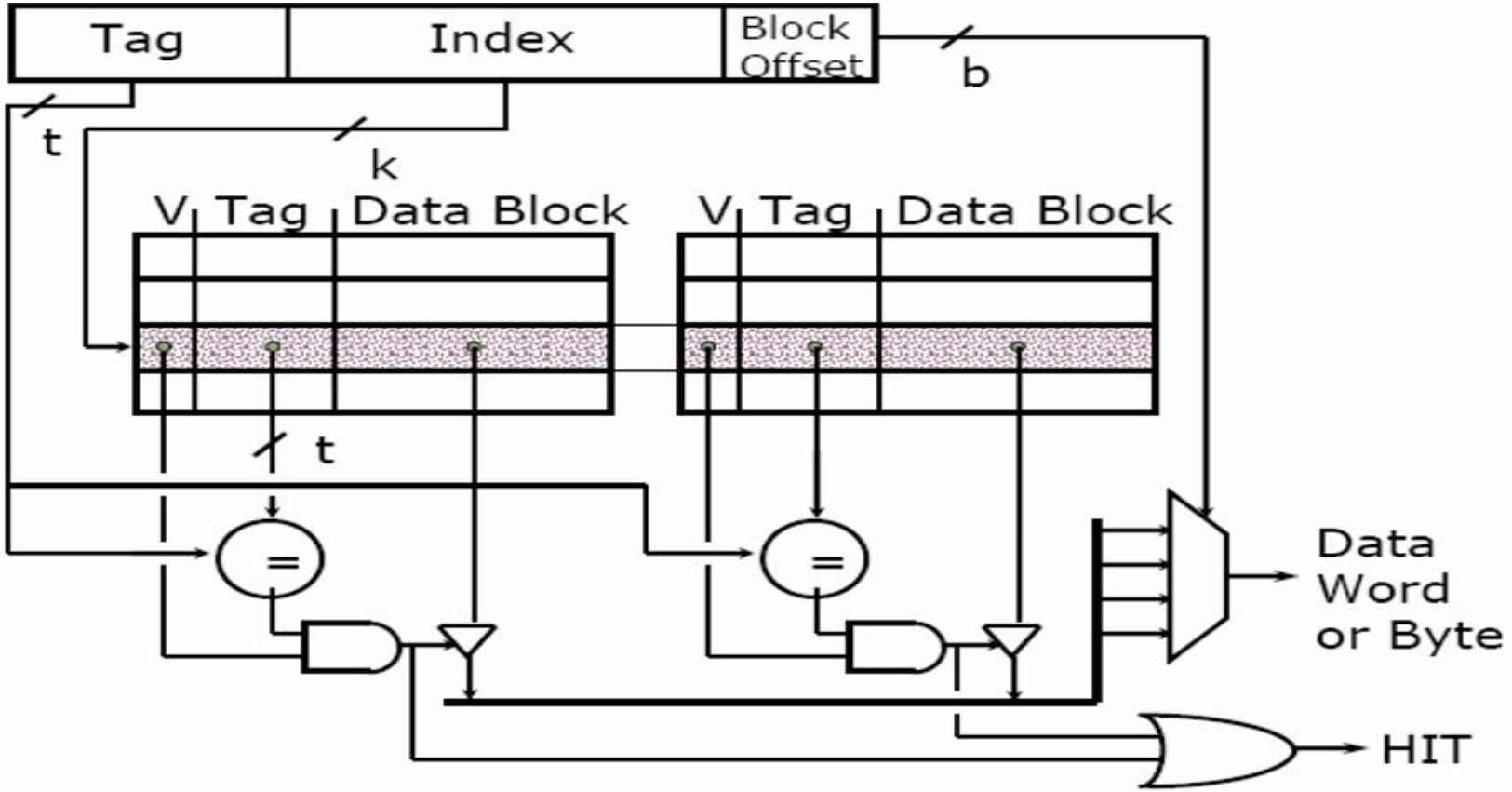


Accessing Set Associative Caches

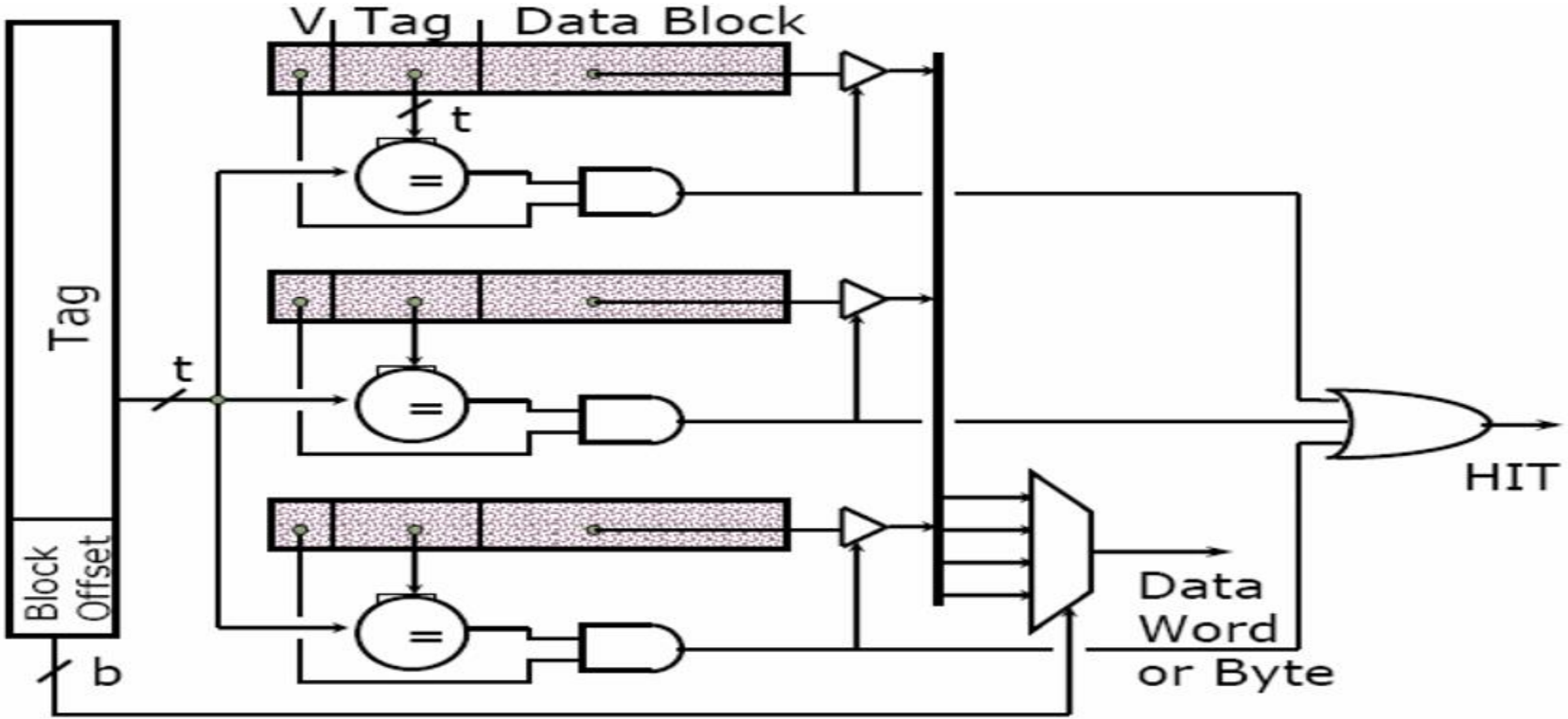
- Word selection is done same as direct mapped cache but chosen only on the line has produced a hit.



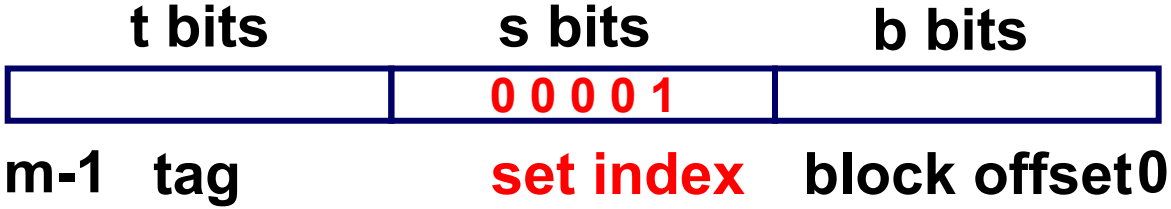
Block Identification – Set Associative



Block Identification – Fully Associative

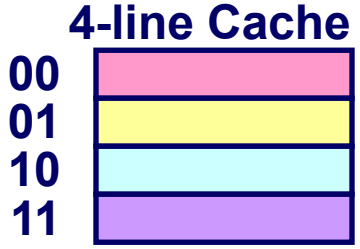


Cache Indexing



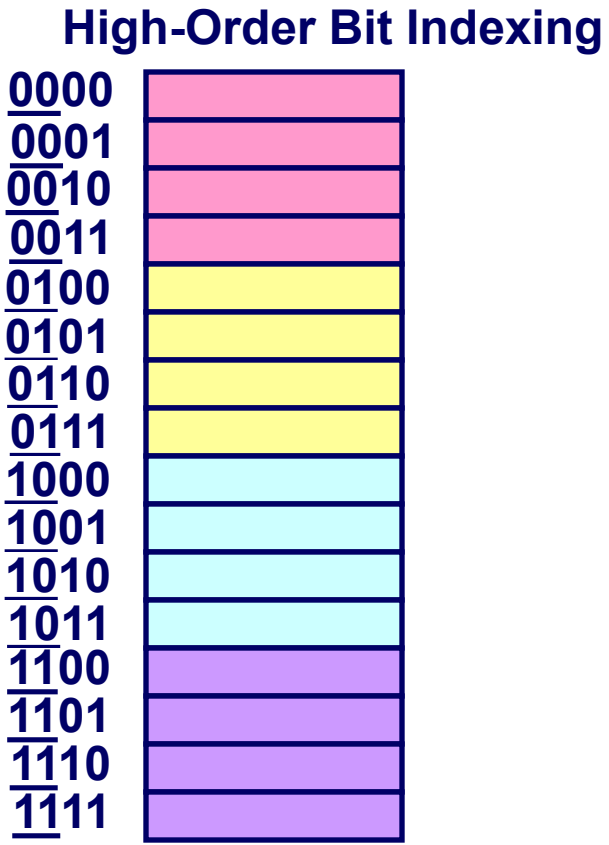
- Decoders are used for indexing
- Indexing time depends on decoder size (s: 2^s)
- Smaller number of sets, less indexing time.

Why Use Middle Bits as Index?

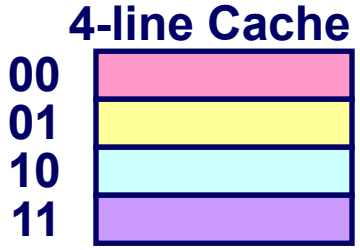


High-Order Bit Indexing

- Adjacent memory lines would map to same cache entry
- Poor use of spatial locality

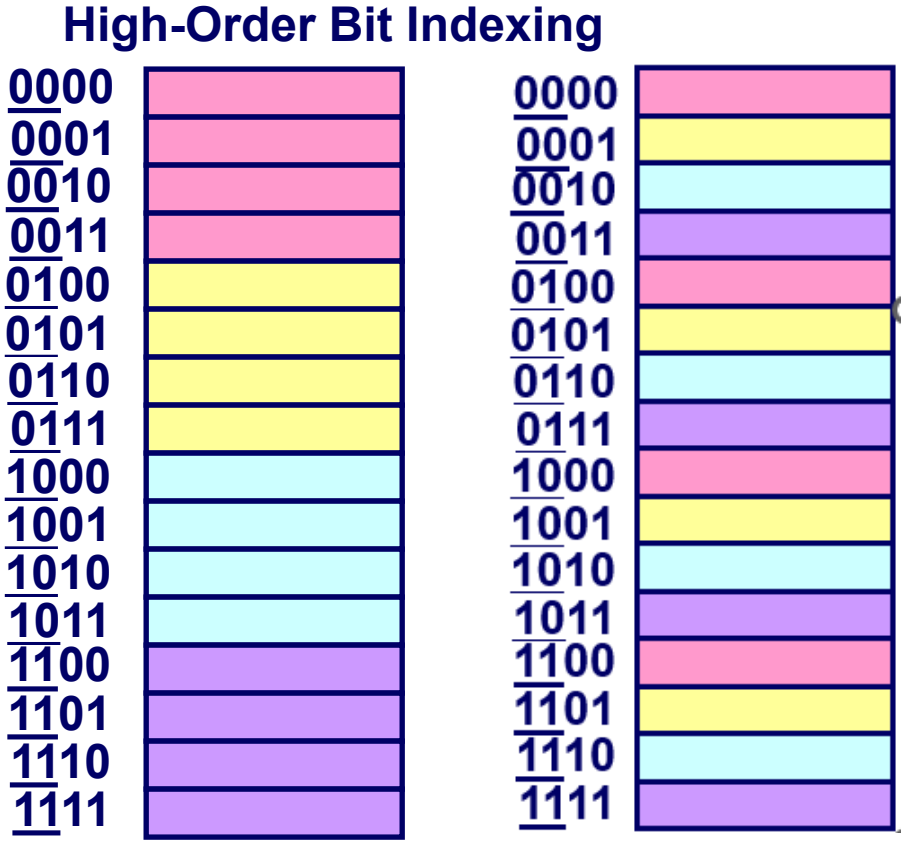


Why Use Middle Bits as Index?



Middle-Order Bit Indexing

- Consecutive memory lines map to different cache lines
- Better use of spacial locality without replacement





johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/